



7

15

1800 SP

عملي

كلية الهندسة المعلوماتية

السنة الثانية

Backtracking & DP

د. عملي مشترك



RB Informatics; 17/12/2024

خوارزميات

ماذا سنتعلم في محاضرة اليوم:

خوارزمية الـ backtracking - Dynamic programming - تمارين .

Backtracking Algorithm

ما هي التراجعية (backtracking) ؟

هي خوارزمية لحل المشكلة بحيث يتم المرور على جميع الحالات الممكنة لحلها بشكل عودي.

يتم تشكيل الحل بشكل تدريجي من خلال اتخاذ قرار و الإكمال على أساسه حتى الوصول إلى نهاية ميتة (dead end) أي عندها يكون الحل إما صحيح أو خاطئ ، ثم يتم التراجع عن هذا القرار و اتخاذ قرار آخر و هكذا حتى تنتهي جميع الخيارات الممكنة.

أنواع المشاكل التي تحلها الـ backtracking :

1. Decision problems : (does the solution exist?)

و هي المشاكل التي تحتاج إلى إيجاد حل فقط بغض النظر إذا كان هذا هو الحل الأمثل أم لا

في هذه الحالة يمكن أن ننهي خوارزمية الـ backtracking عند الوصول إلى حل صحيح ، دون الحاجة إلى المرور على جميع الحالات.

2. Optimization problems : (what is the optimal solution?)

و هي المشاكل التي تتطلب البحث عن الحل الأمثل

في هذه الحالة عند الوصول إلى حل نقوم بتخزينه ، ثم نقارن الحل التالي معه . إذا كان الحل التالي أفضل من الحل السابق فنضعه مكانه ، و إذا لم يكن الحل التالي أفضل من السابق فنتجاهل الحل التالي
ثم نكمل بنفس الأسلوب حتى تنتهي جميع الحلول و بهذا نكون أوجدنا الحل الأمثل.

3. Enumeration problems : (how many solutions exist?)

و هي المشاكل التي تتطلب إيجاد جميع الحلول الممكنة

الشكل العام لخوارزمية الـ backtracking :

```

1. void FIND_SOLUTIONS(parameters){
2.   if (valid solution){
3.     store the solution
4.     return
5.   }
6.   for (all choices){
7.     if (valid choice){
8.       APPLY(choice)
9.       FIND_SOLUTIONS(parameters)
10.      BACKTRACK(remove choice)
11.    }
12.  }
13.  Return
14.}
```

• الشرط (valid solution) if هو الـ base case

، أي حالة التوقف في التابع العودي ، و عندها يتم

التحقق من إذا كان الحل صحيح أم لا (إذا كان

يمثل حل للمشكلة أم لا) ، وإذا كان صحيحاً نقوم

بحفظ الحل بطريقة ما بعد إيجاده ، ويمكن أن لا

نحفظه وهذا حسب المطلوب في المسألة (إيجاد

جميع الحلول... إيجاد الحل الأمثل.....)

- في الحلقة (all choices) for نمر على جميع

الخيارات الممكنة ، و عند كل خيار نقوم بالتالي:

○ نتخذ القرار باعتماد هذا الخيار

○ نستدعي التابع بشكل عودي من أجل القرار الذي تم اتخاذه (بهذا نقوم بإيجاد جميع الحلول الممكنة

المبنية على هذا القرار)

○ نتراجع عن هذا القرار

ثم ننتقل للخيار التالي و نكرر العملية حتى تنتهي جميع الخيارات

التعقيد الزمني (time complexity) لل backtracking :

يمكن تمثيل هذه الخوارزمية بشكل شجرة لها فروع و عمق ، حيث يمكن أن نستخدم القانون :

$\text{Branches}^{\text{Depth}}$

حيث :

Branches : عدد الخيارات التي يمكن اعتمادها في كل مرحلة (عند كل عقدة من الشجرة)

Depth : عدد المراحل التي سيتم المرور بها عند اتخاذ أي قرار حتى الوصول إلى حل نهائي (dead end)

غالباً ما يحسب تعقيد الـ backtrack بهذه الطريقة ولكن لا يشترط أن يكون حساب التعقيد بها دوماً (في بعض الحالات قد لا تكون هذه الطريقة التي نحسب بها التعقيد) .

خوارزمية الـ Greedy & الـ Brute-force :

Brute-force	Greedy	
هي خوارزمية بسيطة تعتمد على إيجاد جميع الحلول الممكنة للمشكلة	هي خوارزمية تقوم باعتماد الخيار الأمثل في كل مرحلة ، حتى تصل إلى حل نهائي للمشكلة	تعريف
- سهلة و غير معقدة - يمكن من خلالها الوصول إلى الحل الأمثل للمشكلة	- تعقيدها الزمني قليل $O(n)$ أو $O(n * \log n)$ (تمثيل الخوارزمية يكون بشكل خطي حيث يتم المرور بمسار واحد فقط اعتماداً على الخيار الأمثل في كل مرحلة) - تصل إلى حل للمشكلة بزمن قليل	إيجابيات
- تعقيدها الزمني عالي ، حيث نقوم بالمرور على جميع الحالات	- ليس بالضرورة أن تصل هذه الخوارزمية إلى الحل الأمثل (فاتخاذ الحل الأمثل في كل مرحلة لا يعني أن الحل النهائي سيكون أمثل أيضاً)	مساوئ
نستخدم هذه الخوارزمية إذا كانت المشكلة تتطلب الوصول إلى جميع الحلول الممكنة ، أو إذا كانت تتطلب الوصول إلى الحل الأمثل	نستخدم هذه الخوارزمية إذا كانت المشكلة تتطلب حل ينشأ عن اتخاذ أفضل قرار في كل مرحلة	استخدام الخوارزمية

خوارزمية الـ backtrack تعتبر أحد أشكال خوارزمية الـ brute-force

تمرين 1 : كتابة تابع يرجع جميع الاحتمالات الممكنة لـ n -bit مكتوبة بالـ binary باستخدام الـ backtracking

مثال للتوضيح :

$n = 2 \rightarrow 00 \ 01 \ 11 \ 10$

$n = 3 \rightarrow 000 \ 001 \ 010 \ 011 \ 100 \ 101 \ 110 \ 111$

الحل :

فكرة التابع :

احتمالات عدد binary من n خانة هي عبارة عن :

0 و على يمينها عدد binary من $n-1$ خانة (أي استدعاء عودي)

1 و على يمينها عدد binary من $n-1$ خانة (استدعاء عودي آخر)

الكود :

```
1. public class Main {
2.     static void generateCode(int n, String result) {
3.         if (n == 0) {
4.             System.out.println(result);
5.             return;
6.         }
7.         generateCode(n - 1, result + "0");
8.         generateCode(n - 1, result + "1");
9.     }
10.    public static void main(String[] args) {
11.        generateCode(2, ""); // OUTPUT : 00 01 10 11
12.        generateCode(3, ""); // OUTPUT : 000 001 010 011 100 101 110 111
13.    }
14. }
```

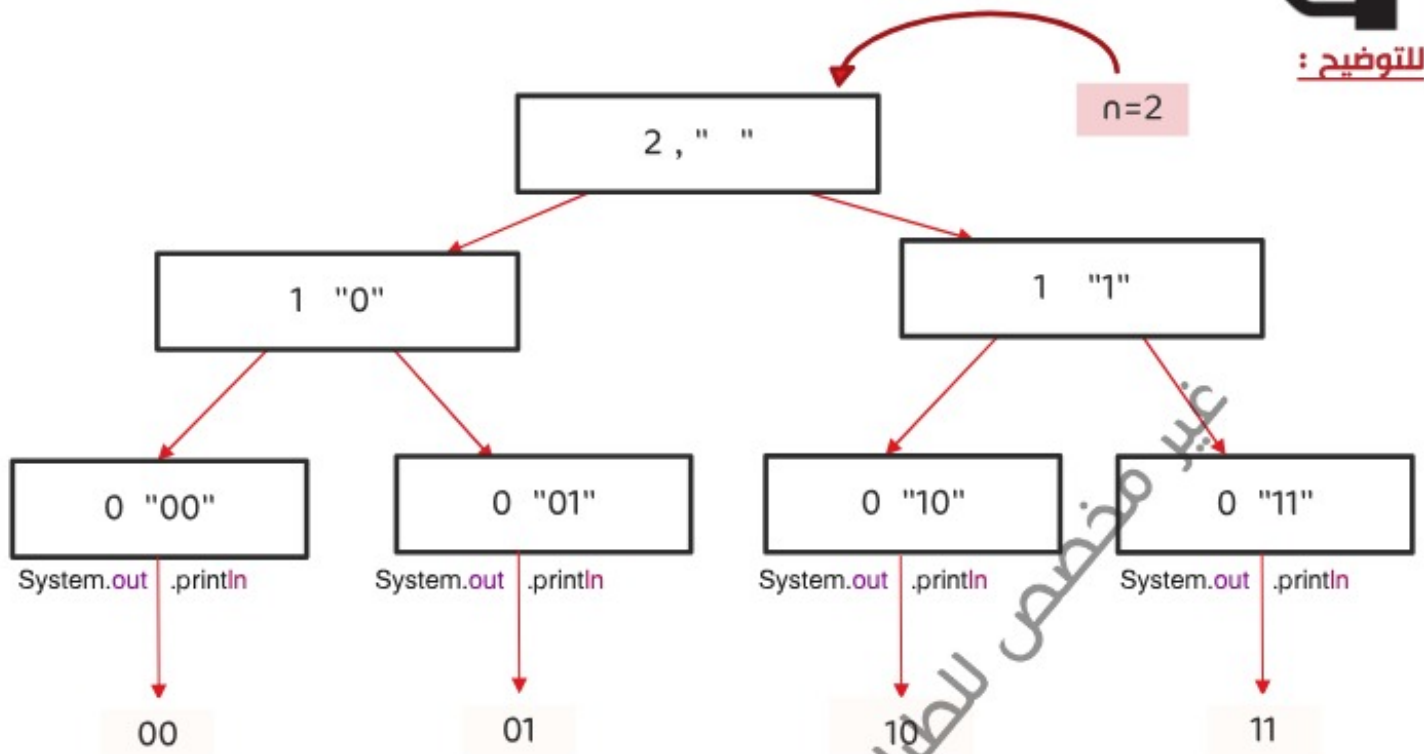
شرح الكود :

- الشرط $n == 0$ ، أي أنه لم يعد هناك خانات لتعبئتها $\leftarrow$ وصلنا إلى dead end

فنقوم بطباعة الحل و return (هذه هي الـ base case)

- استدعاء التابع لإيجاد احتمالات عدد من $n-1$ خانة و على يسارهم 0

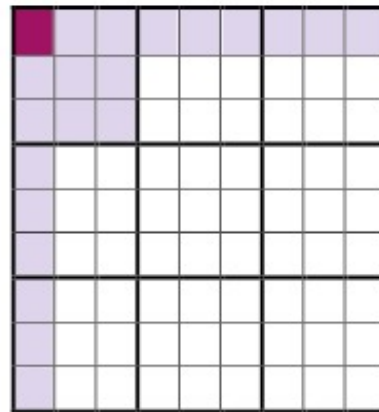
- استدعاء التابع لإيجاد احتمالات عدد من $n-1$ خانة و على يسارهم 1



تمرين 2: تابع يعيد حل رقعة سودوكو 9*9 باستخدام الـ backtracking

قوانين السودوكو :

كل خلية يجب أن تحوي رقم من 1 إلى 9، بحيث أن هذا الرقم غير مكرر في صف هذه الخلية ، أو في عمودها ، أو في الرقعة الجزئية 3*3 التي تحويها



ملاحظة للحل :

سيتم إدخال رقعة السودوكو 9*9 على شكل مصفوفة ثنائية من char ، تحوي على خلايا فيها أرقام و أخرى فارغة يجب تعبئتها ، حيث أن الخلايا الفارغة في الرقعة ستأخذ قيمة '?'

```

public class Main {
1. static void solveSudoku(char[][] board, int i, int j) {
2.     if (j == 9) {
3.         solveSudoku(board, i+1, 0);
4.         return;
5.     }
6.     if (i == 9) {
7.         if (check(board))
8.             printBoard(board);
9.         return;
10.    }
11.    if (board[i][j] != '?') {
12.        solveSudoku(board, i, j+1);
13.        return;
14.    }
15.    for (char k = '1'; k <= '9'; k++) {
16.        board[i][j] = k;
17.        solveSudoku(board, i, j+1);
18.    }
19.    board[i][j] = '?';
20. }
21. static boolean check(char[][] board) {
    // Create arrays to check rows, columns, and
    subgrids
22.    boolean[][] rows = new boolean[9][9];
23.    boolean[][] cols = new boolean[9][9];
24.    boolean[][] subgrids = new boolean[3][3][9];
25.    for (int i = 0; i < 9; i++) {
26.        for (int j = 0; j < 9; j++) {
27.            if (board[i][j] != '?') {
28.                int num = board[i][j] - '1'; // convert
char to index (0-8)
29.                // Check the row
30.                if (rows[i][num]) {
31.                    return false;
32.                }

```

```

33. rows[i][num] = true;
34.        // Check the column
35.        if (cols[j][num]) {
36.            return false;
37.        }
38.        cols[j][num] = true;
39.        // Check the subgrid
40.        int subgridRow = i / 3;
41.        int subgridCol = j / 3;
42.        if (subgrids[subgridRow][subgridCol][num]) {
43.            return false;
44.        }
45.        subgrids[subgridRow][subgridCol][num] = true;
46.    }
47.    }
48.    }
49.    return true;
50. }
51. static void printBoard(char[][] board) {
52.    for (int i = 0; i < 9; i++) {
53.        for (int j = 0; j < 9; j++) {
54.            System.out.print(board[i][j] + " ");
55.            if (j%3 == 2)
56.                System.out.print(" ");
57.        }
58.        System.out.println();
59.        if (i%3 == 2)
60.            System.out.println();
61.    }
62. }

```

```

63. public static void main(String[] args) {
64.    char[][] board = {{ '2', '3', '4', '6', '7', '8', '9', '1', '2' }, { '6', '7', '2', '1', '9', '5', '3', '4', '8' }, { '1', '9', '8', '3', '4', '2', '5', '6', '7' },
                        { '8', '5', '9', '7', '6', '1', '4', '2', '3' }, { '4', '2', '6', '8', '5', '3', '7', '9', '1' }, { '7', '1', '3', '9', '2', '4', '8', '5', '6' },
                        { '9', '6', '1', '5', '3', '7', '2', '8', '4' }, { '2', '8', '7', '4', '1', '9', '6', '3', '5' }, { '3', '4', '5', '2', '8', '6', '1', '7', '9' } };
65.    solveSudoku (board, 0, 0); }

```

لا يوجد بعض التوابع المساعدة، لنفهمها سوياً :

1. `check (char [][] board) :`

هذا التابع يقوم بالتأكد أن المصفوفة الممررة إليه (board) صحيحة أم لا (تحقق قواعد السودوكو أم لا) .
حيث يقوم بتقسيم الرقعة إلى مصفوفة rows تحوي جميع سطورها، و مصفوفة cols تحوي جميع أعمدها، و مصفوفة subgrids التي تحوي الرقع الجزئية (3*3) .
ثم يمر على جميع خلايا الرقعة الكلية و يتأكد من أن كل خلية غير مكررة بنفس الصف ، أو العمود ، أو الرقعة الجزئية
يعيد التابع true إذا كانت الرقعة صحيحة و false إذا خاطئة .

2. `printBoard (char [][] board) :`

هذا التابع يقوم بطباعة المصفوفة الممررة board بشكل يسهل قرائته، حيث يفصل بين كل رقعة جزئية و الأخرى

لا الآن لنشرح تابع solveSudoku و هو حل التمرين:

▪ فكرة التابع :

سيقوم بالمرور على جميع خلايا الرقعة ، إذا كانت الخلية تحوي رقم فسيجاهلها و ينتقل للاستدعاء التالي ، أما إذا كانت الخلية تحوي '?' فسيقوم بملئ هذه الخلية بعدد بدأ من 1 إلى 9 ثم الانتقال إلى الخلية التالية حتى يصل إلى نهاية الرقعة عندها سيتم التأكد أنه إذا كان حل الرقعة صحيح أم لا ، في حال كان الحل خاطئ فسيتم التراجع عن آخر قرار تم اتخاذه (يعيد آخر خلية فارغة لتساوي '?') ليأخذ قرار آخر و يعيد العملية فيتم بذلك تجربة جميع الحالات الممكنة حتى نصل إلى حل للرقعة .

▪ ال parameters :

1. `char [][] board` : و هي رقعة السودوكو

2. `int i` : رقم سطر الخلية الحالية التي نعالجها

3. `int z` : رقم عمود الخلية الحالية التي نعالجها



شرح الكود :

- الشرط $z==9$ يعني أن الخلية الحالية هي آخر خلية في السطر ، فنقوم بالانتقال إلى السطر التالي.
- الشرط $i==9$ يعني أن الخلية الحالية هي الخلية الأخيرة في الرقعة (لأن هذا الشرط يعالج حالة آخر خلية في العمود ، و الشرط السابق عالج حالة آخر خلية في السطر)
- هنا نقوم بالتحقق من إذا كانت الرقعة صحيحة أم لا و return (هذه هي الـ base case)
- الشرط $board[i][j] != '?'$ يعني أن الخلية الحالية تحوي رقم ، فنقوم بالانتقال إلى الخلية التي تليها في السطر (أي أن الانتقال يكون بشكل أفقي في المصفوفة ، حتى نصل إلى نهاية السطر أي يتحقق الشرط $z==9$ فننتقل إلى السطر التالي).
- الحلقة for نصل إليها عندما لا تتحقق الشروط السابقة ، أي ستكون الخلية تحوي '?' ، داخل الحلقة :
نقوم بوضع العدد k (قيمته تبدأ من 1 و تنتهي بـ 9) داخل الخلية بدلاً من ؟ ، و عند كل قيمة نستدعي التابع من أجل الخلية التالية ليقوم بنفس المراحل السابقة .
- عند الانتهاء من جميع الاحتمالات ($9 \leftarrow 1$) للخلية $[i][j]$ نقوم بالتراجع و نعيد الخلية لتساوي '?'

في الـ main :

- مثال بسيط عن رقعة سودوكو فيها خلية واحدة فارغة .
- فيكون الـ Output :



5 3 4	6 7 8	9 1 2
6 7 2	1 9 5	3 4 8
1 9 8	3 4 2	5 6 7
8 5 9	7 6 1	4 2 3
4 2 6	8 5 3	7 9 1
7 1 3	9 2 4	8 5 6
9 6 1	5 3 7	2 8 4
2 8 7	4 1 9	6 3 5
3 4 5	2 8 6	1 7 9

طريقة أربيسيزية أخرى للحل

```

public class SudokuSolver {
    static final int N = 9;
    // A utility function to print grid
    static void print(char[][] grid) {
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                System.out.print(grid[i][j] + " ");
            }
            System.out.println();
        }
    }
    // Checks whether it will be legal to assign num to the given row, col
    static boolean isSafe(char[][] grid, int row, int col, char num) {
        // Check if we find the same num in the similar row, we return false
        for (int x = 0; x < N; x++) {
            if (grid[row][x] == num) {
                return false;
            }
        }
        // Check if we find the same num in the similar column, we return false
        for (int x = 0; x < N; x++) {
            if (grid[x][col] == num) {
                return false;
            }
        }
        int startRow = row - row % 3;
        int startCol = col - col % 3;
        // Check if we find the same num in the particular 3*3 matrix, we return false
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                if (grid[i + startRow][j + startCol] == num) {
                    return false;
                }
            }
        }
        return true;
    }
    // Takes a partially filled-in grid and attempts to assign values to all unassigned
    locations in such a way to meet the requirements for Sudoku solution (non-duplication
    across rows, columns, and boxes)

```

```
static boolean solveSudoku(char[][] grid, int row, int col) {
    // Check if we have reached the 8th row and 9th column (0 indexed matrix), we are
    // returning true to avoid further backtracking
    if (row == N - 1 && col == N) {
        return true;
    }
    // Check if column value becomes 9, we move to the next row and column starting
    // from 0
    if (col == N) {
        row++;
        col = 0;
    }
    // Check if the current position of the grid already contains value >0, we iterate for
    // the next column
    if (grid[row][col] != '?') {
        return solveSudoku(grid, row, col + 1);
    }
    for (char num = '1'; num <= '9'; num++) {
        // Check if it is safe to place the num (1-9) in the given row, col -> we move to the
        // next column
        if (isSafe(grid, row, col, num)) {
            // Assigning the num in the current (row,col) position of the grid and assuming
            // our assigned num in the position is correct
            grid[row][col] = num;
            // Checking for the next possibility with the next column
            if (solveSudoku(grid, row, col + 1)) {
                return true;
            }
            // Removing the assigned num, since our assumption was wrong, and we go for
            // the next assumption with a different num value
            grid[row][col] = '?';
        }
        return false;
    }
}

public static void main(String[] args) {
    char[][] grid = {
        {'3', '?', '6', '5', '?', '8', '4', '?', '?'}, {'5', '2', '?', '?', '?', '?', '?', '?', '?'},
        {'?', '8', '7', '?', '?', '?', '?', '3', '1'}, {'?', '?', '3', '?', '1', '?', '?', '8', '?'},
        {'9', '?', '?', '8', '6', '3', '?', '?', '5'}, {'?', '5', '?', '?', '9', '?', '6', '?', '?'},
        {'1', '3', '?', '?', '?', '?', '2', '5', '?'}, {'?', '?', '?', '?', '?', '?', '?', '7', '4'},
        {'?', '?', '5', '2', '?', '6', '3', '?', '?'}
    };
    if (solveSudoku(grid, 0, 0)) {
        print(grid);
    } else { System.out.println("No solution exists.");
    }
}
```

Dynamic Programming (DP)

ما هو الـ Dynamic Programming (DP) ؟

هو طريقة لحل المشاكل من خلال تقسيم المشكلة إلى مشاكل جزئية صغيرة ، ثم تخزين نتائج هذه المشاكل الجزئية لتجنب إعادة حسابها بشكل متكرر

أنواعه:

1. Top-Down (Memorization)

هنا نقوم بحل المشكلة بشكل عودي و نخزن النتائج لتجنب حساب الاستدعاءات العودية بشكل متكرر (سيكون تمثيل التابع كشجرة ، حيث أننا نبدأ بالمشكلة الكبيرة و نتفرع إلى مشاكل أصغر) .

2. Bottom-Up (Tabulation)

هنا نبدأ بحل أصغر المشاكل الجزئية و بالاعتماد على نتائجهم نقوم ببناء حل المشكلة الكبيرة بشكل تكراري (أي أننا سنبدأ من نهاية الشجرة و نبني المشاكل بشكل تدريجي حتى نصل إلى بدايتها)

تمرين 3: تابع يعيد Longest Increasing Subsequence (LIS)، أي يعيد طول أطول سلسلة تصاعدية

مثلا : لدينا سلسلة الأرقام:

10 - 5 - 2 - 7 - 1 - 10

فأطول سلسلة تصاعدية هي :

10 - 5 - 2 - 1

أي أن الخرج يكون : 4

■ ملاحظة :

المرور على الأرقام يتم من اليسار إلى اليمين ، أي لا يمكننا اختيار رقم يسبق الرقم الحالي



الحل باستخدام الـ backtracking:

```

1. int backtrackLIS(int[] arr, int i, int j) {
2.   if (i == arr.length) return 0;
3.   int exclude = backtrackLIS(arr, i + 1, j);
4.   int include = 0;
5.   if (j == -1 || arr[i] >= arr[j]) {
6.     include = backtrackLIS(arr, i + 1, i) + 1;
7.   }
8.   return Math.max(include, exclude);
9. }

```

فكرة التابع:

عند كل عنصر يوجد خيارين، إما أن نضيف العنصر الحالي للسلسلة وتكون الإضافة بزيادة قيمة include بمقدار 1 و نكمل البحث عن الأعداد الأكبر منه ، أو أن نتجاهل العنصر و ننتقل إلى العنصر الذي يليه فنقوم بنفس الخيارين . ثم نعيد السلسلة الأطول من هذين الخيارين

الـ parameters:

1. int[] arr : هي مصفوفة الأعداد الكلية

2. int i : هو index العنصر الحالي

3. int j : هو index آخر عنصر تمت إضافته إلى سلسلة الأرقام التصاعدية (سنعطيه قيمة ابتدائية -1 تدل على أنه لم يتم إضافة عناصر بعد)

شرح الكود:

- الشرط $i == arr.length$ هو الـ base case حيث نكون قد وصلنا إلى نهاية المصفوفة فنقوم بـ return 0
- حالة الـ exclude : هنا نقوم بتجاهل العنصر $arr[i]$ ، حيث نقوم باستدعاء عودي للتابع من أجل $i+1$ و ز ذاتها . لحساب LIS لهذه الحالة و تخزينه
- حالة الـ include : هنا نقوم بإضافة العنصر $arr[i]$ إلى السلسلة ، تتم هذه الحالة فقط إذا كان العنصر $arr[i]$ أكبر من $arr[j]$ (و هو آخر عنصر في السلسلة التصاعدية) ، أو إذا كان $j == -1$ و (هي الحالة الابتدائية لـ j) ، نقوم هنا باستدعاء عودي للتابع من أجل $i+1$ (أي ننتقل إلى العنصر التالي) ، و قيمة ز تصبح i (لأننا أضفنا $arr[i]$ إلى السلسلة التصاعدية فيكون هو آخر عنصر تمت إضافته) ، و نضيف لنتائج الاستدعاء 1 لأننا قمنا بإضافة عنصر و نخزن الناتج .
- في نهاية التابع سنقارن العددين $include$ و $exclude$ و نعيد الأكبر بينهما (أي سنعيد الطول الأكبر)

التعقيد : $O(2^n)$

الحل باستخدام الـ Top-Down (memorization) :

فكرة الحل

```

1. int topDownLIS(int[] arr, int i, int j) {
2.   if (dp[i][j+1] != -1) return dp[i][j+1];
3.   if (i == arr.length) return 0;
4.   int exclude = topDownLIS(arr, i+1, j);
5.   int include = 0;
6.   if (j == -1 || arr[i] >= arr[j]) {
7.     include = topDownLIS(arr, i+1, i) + 1;
8.   }
9.   dp[i][j+1] = Math.max(include, exclude);
10.  return dp[i][j+1];
11. }

```

هي نفس طريقة الـ backtracking لكن هنا ننشئ مصفوفة dp بحيث نعطي جميع عناصرها القيمة -1 في البداية

ثم في التابع نقوم بالتالي :

- إذا كان العنصر $dp[i][j+1]$ يساوي الـ -1 فنقوم بحسابه و تخزينه (أي نقوم بالخطوات السابقة في حل الـ

backtracking مع تخزين القيمة النهائية في الـ dp)

- إذا كان العنصر لا يساوي -1 فهذا يعني أننا مررنا على هذا الاستدعاء من قبل وقمنا بتخزينه ، فلا داعي لاستدعائه مرة أخرى فقط نعيد قيمته .

الحل باستخدام الـ Bottom-Up (Tabulation) : (طريقة إضافية)

فكرة الكود :

```

1. int bottomUpLIS(int[] arr)
2. int n = arr.length;
3. int[] dp = new int[n];
4. Arrays.fill(dp, -1);
5. for (int i = 1; i < n; i++) {
6.   for (int j = 0; j < i; j++) {
7.     if (arr[i] > arr[j]) {
8.       dp[i] = Math.max(dp[i], dp[j] + 1);
9.     }
10.  }
11. int maxLength = 0;
12. for (int val : dp)
13.   maxLength = Math.max(maxLength, val);
14. return maxLength;

```

سنحل المشكلة بطريقة مختلفة ، حيث سنقارن كل عنصر (بدءاً من العنصر $arr[1]$) بالعناصر التي قبله و نخزن طول أطول سلسلة حيث تكون جميع العناصر السابقة مرتبة تصاعدياً

شرح الكود :

- نعرف مصفوفة dp و نعطي جميع عناصرها القيمة 1
- الحلقة الخارجية تمر على جميع عناصر

المصفوفة.

- الحلقة الداخلية تمر على جميع العناصر الأصغر من $arr[i]$ ، داخلها :

إذا تحقق الشرط $arr[i] > arr[j]$ فنقوم بإسناد قيمة أطول سلسلة تصاعدية بين العنصرين $arr[i]$ و $arr[0]$ في

الـ $dp[i]$ ، في حال لم يتحقق سيبقى الـ $dp[i] = 1$.

- عند انتهاء الحلقتين ستكون الـ dp عبارة عن مصفوفة أطوال السلاسل التصادية

- ثم نقوم بطباعة أطول عنصر في هذه السلسلة .

مثال للتوضيح : بفرض لدينا المصفوفة :

$\{3, 10, 2, 1, 20\}$

داخل التابع سيتم التالي :

■ $dp = \{1, 1, 1, 1, 1\}$:

$i = 1 \Rightarrow j = 0 \rightarrow 10 > 3 \rightarrow dp[1] = \max(dp[1], dp[0] + 1) \rightarrow dp[1] = \max(1, 2) = 2 \rightarrow dp = \{1, 2, 1, 1, 1\}$

■ $dp = \{1, 2, 1, 1, 1\}$:

$i = 2 \Rightarrow j = 0 \rightarrow 2 > 3 \text{ X}$

$\Rightarrow j = 1 \rightarrow 2 > 10 \text{ X} \rightarrow dp = \{1, 2, 1, 1, 1\}$

■ $dp = \{1, 2, 1, 1, 1\}$:

$i = 3 \Rightarrow j = 0 \rightarrow 1 > 3$

$\Rightarrow j = 1 \rightarrow 1 > 10 \text{ X}$

$\Rightarrow j = 2 \rightarrow 1 > 2 \text{ X} \rightarrow dp = \{1, 2, 1, 1, 1\}$

■ $dp = \{1, 2, 1, 1, 1\}$:

$i = 4 \Rightarrow j = 0 \rightarrow 20 > 3 \rightarrow dp[4] = \max(dp[4], dp[0] + 1) \rightarrow dp[1] = \max(1, 2) = 2 \rightarrow dp = \{1, 2, 1, 1, 2\}$

$\Rightarrow j = 1 \rightarrow 20 > 10 \rightarrow dp[4] = \max(dp[4], dp[1] + 1) \rightarrow dp[1] = \max(2, 3) = 3 \rightarrow dp = \{1, 2, 1, 1, 3\}$

$\Rightarrow j = 2 \rightarrow 20 > 2 \rightarrow dp[4] = \max(dp[4], dp[2] + 1) \rightarrow dp[1] = \max(3, 2) = 3 \rightarrow dp = \{1, 2, 1, 1, 3\}$

$\Rightarrow j = 3 \rightarrow 20 > 1 \rightarrow dp[4] = \max(dp[4], dp[3] + 1) \rightarrow dp[1] = \max(3, 2) = 3 \rightarrow dp = \{1, 2, 1, 1, 3\}$

$dp = \{1, 2, 1, 1, 3\} \rightarrow \text{maximum value in } dp \text{ is } 3 \rightarrow \text{output : } 3$

الفرق بين الـ backtracking و الـ DP :

- نختار الـ backtracking عندما تتطلب المشكلة المرور على جميع الاحتمالات الممكنة للحل ، و لا تحتوي على مشاكل جزئية متداخلة أو مكررة (أي عندما يكون تجزيء المشكلة إلى مشاكل صغيرة غير فعال) .

مثل : decision problems, Enumeration problems

- نختار الـ DP عندما تكون المشكلة مقسمة إلى مشاكل صغيرة متداخلة أو مكررة

مثل : optimization problems

DP	backtracking	
تعتمد على الـ recursion أو الـ iteration	تعتمد على الـ recursion	البنية
تتجنب تكرار حساب الحالة	تجرب كل الحالات	طريقة العمل
array (حيث يتم حفظ المعلومات)	stack (لأنه recursion)	استهلاك الذاكرة
Polynomial	Exponential	التعقيد الزمني
أسرع و أكثر كفاءة	بطيء جداً عند المشاكل الكبيرة	الكفاءة
LIS	N-Queen, Sudoku	مثال



Ask not what your country can do for you; ask what you can do for your country.

John Kennedy

انتهت المحاضرة....